



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе № 3

по курсу «Защита информации»

на тему: «Разработка алгоритма шифрования с открытым ключом»

Студент ИУ7-71Б
(Группа)

(Подпись, дата)

Я. Р. Овчинников
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Ю. С. Руденкова
(И. О. Фамилия)

2025 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ	3
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	4
РЕАЛИЗАЦИЯ	8
ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ	12

ЗАДАНИЕ

1. Цель работы

Разработка алгоритма шифрования с открытым ключом. Шифрование и расшифровка архивных файлов.

2. Задание

Реализовать программу шифрования алгоритмом с открытым ключом. Использовать алгоритм RSA. Обеспечить шифрование и расшифровку файла архива (rar, zip или др.) с использованием разработанной программы.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1. Алгоритм шифрования с открытым ключом архивного файла

Алгоритм RSA шифрования архивного файла включает следующие этапы:

1) Подготовка данных:

- Чтение архивного файла (rar, zip и др.) в двоичном режиме;
- Определение размера файла и количества блоков для обработки;
- Вычисление максимального размера блока (на 1 байт меньше размера модуля n для обеспечения условия $m < n$).

2) Разбиение на блоки:

- Файл разбивается на блоки фиксированного размера;
- Последний блок может быть меньше остальных;
- Размер блока определяется разрядностью модуля: $\text{block_size} = \lfloor (\text{bits}(n) - 1) / 8 \rfloor$ байт.

3) Шифрование каждого блока:

- Преобразование блока байтов в большое целое число m : $m = \sum_{i=0}^{k-1} \text{byte}_i \cdot 256^{k-1-i}$;
- Проверка условия $m < n$;
- Выполнение операции шифрования: $c = m^e \bmod n$;
- Преобразование результата c обратно в последовательность байтов фиксированной длины.

4) Формирование выходного файла:

- Запись заголовка с размером зашифрованного блока;
- Последовательная запись всех зашифрованных блоков;
- Все зашифрованные блоки имеют одинаковый размер, равный размеру модуля n в байтах.

2. Алгоритм расшифровки с открытым ключом архивного файла

Процесс расшифровки является обратным к шифрованию:

1) Чтение заголовка:

- Извлечение размера зашифрованного блока из заголовка файла;
- Проверка корректности формата зашифрованного файла.

2) Обработка зашифрованных блоков:

- Последовательное чтение блоков фиксированного размера;
- Каждый блок имеет размер, равный размеру модуля n в байтах.

3) Расшифровка каждого блока:

- Преобразование блока байтов в большое целое число c ;
- Выполнение операции расшифрования: $m = c^d \bmod n$;
- Преобразование числа m обратно в последовательность байтов переменной длины (без ведущих нулей).

4) Запись расшифрованных данных:

- Последовательная запись расшифрованных блоков в выходной файл;
- Восстановление исходного архивного файла.

3. Асимметричное шифрование RSA

RSA (Rivest–Shamir–Adleman) — это алгоритм асимметричного шифрования с открытым ключом, основанный на вычислительной сложности задачи факторизации больших целых чисел.

Определение асимметричного шифрования: Система шифрования, в которой для шифрования и расшифровки используются различные ключи — публичный (открытый) и приватный (секретный). Знание публичного ключа не позволяет вычислить приватный ключ за приемлемое время.

Математические основы RSA:

1) Генерация ключей:

- Выбираются два больших простых числа p и q (в данной реализации — по 256 бит каждое);
- Вычисляется модуль: $n = p \cdot q$ (512 бит);
- Вычисляется функция Эйлера: $\varphi(n) = (p - 1)(q - 1)$;
- Выбирается публичная экспонента e (обычно 65537), такая что $\gcd(e, \varphi(n)) = 1$;
- Вычисляется приватная экспонента: $d = e^{-1} \bmod \varphi(n)$.

2) **Публичный ключ:** (n, e) — используется для шифрования

3) **Приватный ключ:** (n, d) — используется для расшифровки

4) Операции шифрования/расшифровки:

- Шифрование: $c = m^e \bmod n$
- Расшифровка: $m = c^d \bmod n$

Кто создаёт ключи?

Получатель сообщения создаёт пару ключей:

- Генерирует публичный и приватный ключи;
- **Публичный ключ** (n, e) распространяется открыто и используется **отправителем** для шифрования сообщений;
- **Приватный ключ** (n, d) хранится в секрете у получателя и используется только им для расшифровки;
- Отправитель шифрует данные публичным ключом получателя, после чего только получатель может их расшифровать своим приватным ключом.

Преимущества асимметричного шифрования:

- Не требуется защищённый канал для обмена ключами;
- Один публичный ключ может использоваться множеством отправителей;
- Обеспечивается конфиденциальность без предварительного согласования секретного ключа.

4. Реализация алгоритма RSA

В данной работе реализована консольная программа для шифрования и расшифровки архивных файлов с использованием алгоритма RSA. Программа включает следующие основные компоненты:

- **Класс RSA** — реализует криптографические операции (генерация ключей, шифрование/расшифровка блоков);
- **Класс ConsoleInterface** — обеспечивает взаимодействие с пользователем через меню;
- **Модуль исключений** — определяет специализированные классы ошибок для различных ситуаций;
- **Библиотека GMP** — используется для работы с большими целыми числами.

Программа поддерживает генерацию 512-битных ключей, их сохранение/загрузку из файлов, а также шифрование и расшифровку файлов любого типа, включая архивы (rar, zip и др.).

РЕАЛИЗАЦИЯ

На листинге 1 приведена реализация функции генерации RSA ключей.

Листинг 1 – Генерация RSA ключей

```
1 void RSA::generate_keys(int bit_length) {
2     std::cout << "Генерация ключей RSA (" << bit_length << "
3         бит)...\n";
4     // Генерируем два простых числа
5     std::cout << "Генерация первого простого числа p..." <<
6         std::flush;
7     mpz_class p = generate_prime(bit_length / 2);
8     std::cout << " готово\n";
9     std::cout << "Генерация второго простого числа q..." <<
10         std::flush;
11     mpz_class q = generate_prime(bit_length / 2);
12     std::cout << " готово\n";
13
14     // Вычисляем n = p * q
15     n = p * q;
16     // Вычисляем функцию Эйлера (n) = (p-1)(q-1)
17     mpz_class phi = (p - 1) * (q - 1);
18     // Выбираем публичную экспоненту e (обычно 65537)
19     e = 65537;
20
21     // Проверяем, что gcd(e, phi) = 1
22     if (gcd(e, phi) != 1) {
23         e = 3;
24         while (gcd(e, phi) != 1) {
25             e += 2;
26         }
27     }
28
29     // Вычисляем приватную экспоненту d
30     std::cout << "Вычисление приватного ключа..." << std::flush;
31     d = mod_inverse(e, phi);
32     std::cout << " готово\n";
33
34     keys_generated = true;
35     std::cout << "Ключи успешно сгенерированы!\n";
36 }
```


На листинге 2 приведена реализация функции генерации простого числа заданной разрядности.

Листинг 2 – Генерация простого числа

```
1  mpz_class RSA::generate_prime(int bits) {
2      gmp_randclass rng(gmp_randinit_default);
3      auto seed =
4          std::chrono::system_clock::now().time_since_epoch().count();
5      rng.seed(seed);
6
7      mpz_class candidate;
8      int max_attempts = 1000;
9
10     for (int attempt = 0; attempt < max_attempts; ++attempt) {
11         // Генерируем случайное нечетное число нужного размера
12         candidate = rng.get_z_bits(bits);
13
14         // Устанавливаем старший бит в 1 (для правильного
15             размера)
16         mpz_setbit(candidate.get_mpz_t(), bits - 1);
17
18         // Делаем число нечетным
19         if (mpz_even_p(candidate.get_mpz_t())) {
20             candidate += 1;
21         }
22
23         if (is_prime(candidate)) {
24             return candidate;
25         }
26     }
27     throw RSAExceptions::PrimeGenerationError();
28 }
```

На листинге 3 приведена реализация функции шифрования блока данных.

Листинг 3 – Шифрование блока

```
1  std::vector<uint8_t> RSA::encrypt_block(const
    std::vector<uint8_t>& block) {
```

```

2 // Преобразуем байты в большое число
3 mpz_class m = bytes_to_mpz(block);
4
5 // Проверяем, что m < n
6 if (m >= n) {
7     throw RSAExceptions::EncryptionError("Блок данных больше
8         модуля");
9 }
10
11 // Шифруем: c = m^e mod n
12 mpz_class c;
13 mpz_powm(c.get_mpz_t(), m.get_mpz_t(), e.get_mpz_t(),
14     n.get_mpz_t());
15
16 // Преобразуем обратно в байты
17 size_t encrypted_size = (mpz_sizeinbase(n.get_mpz_t(), 2) +
18     7) / 8;
19 return mpz_to_bytes(c, encrypted_size);
20 }

```

На листинге 4 приведена реализация функции расшифровки блока данных.

Листинг 4 – Расшифровка блока

```
1 std::vector<uint8_t> RSA::decrypt_block(const
    std::vector<uint8_t>& block) {
2     // Преобразуем байты в большое число
3     mpz_class c = bytes_to_mpz(block);
4     // Расшифровываем:  $m = c^d \bmod n$ 
5     mpz_class m;
6     mpz_powm(m.get_mpz_t(), c.get_mpz_t(), d.get_mpz_t(),
        n.get_mpz_t());
7     // Преобразуем обратно в байты (без дополнительных нулей)
8     size_t decrypted_size =
        (mpz_sizeinbase(m.get_mpz_t(), 2) + 7) / 8;
9     if (decrypted_size == 0) decrypted_size = 1;
10    return mpz_to_bytes(m, decrypted_size);
11 }
```

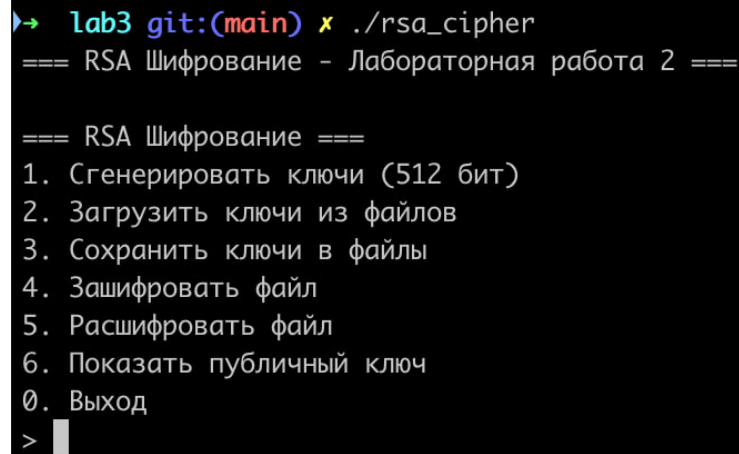
На листинге 5 приведена реализация вспомогательных функций преобразования данных.

Листинг 5 – Преобразование между байтами и большими числами

```
1 mpz_class RSA::bytes_to_mpz(const std::vector<uint8_t>& bytes) {
2     mpz_class result = 0;
3     for (size_t i = 0; i < bytes.size(); ++i) {
4         result = (result << 8) | bytes[i];
5     }
6     return result;
7 }
8 std::vector<uint8_t> RSA::mpz_to_bytes(const mpz_class& num,
    size_t size) {
9     std::vector<uint8_t> bytes(size);
10    mpz_class temp = num;
11    for (int i = size - 1; i >= 0; --i) {
12        bytes[i] = mpz_get_ui(temp.get_mpz_t()) & 0xFF;
13        temp >>= 8;
14    }
15    return bytes;
16 }
```

ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ

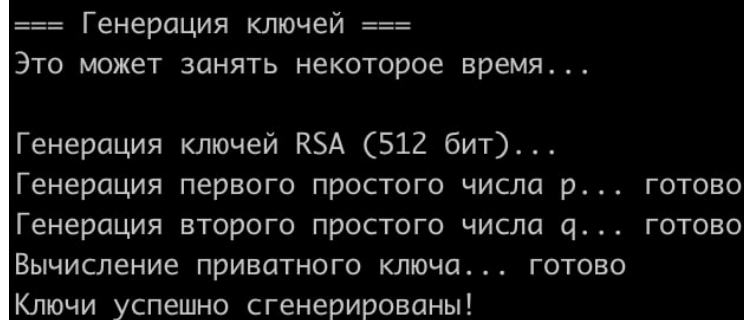
На картинках ниже продемонстрированы основные сценарии работы программы



```
lab3 git:(main) x ./rsa_cipher
=== RSA Шифрование - Лабораторная работа 2 ===

=== RSA Шифрование ===
1. Сгенерировать ключи (512 бит)
2. Загрузить ключи из файлов
3. Сохранить ключи в файлы
4. Зашифровать файл
5. Расшифровать файл
6. Показать публичный ключ
0. Выход
>
```

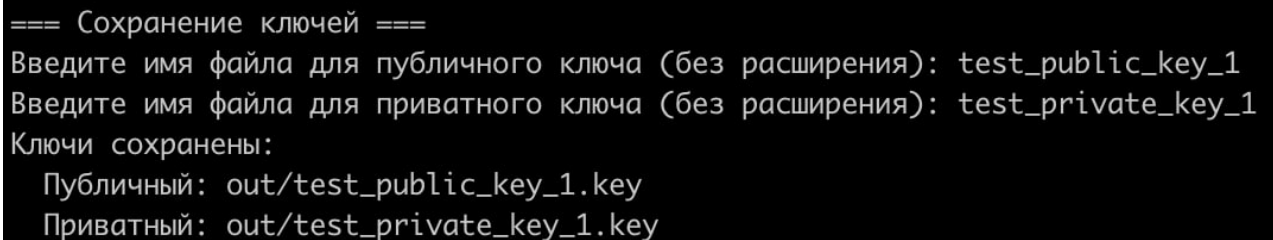
Рисунок 1 – Основное меню программы



```
=== Генерация ключей ===
Это может занять некоторое время...

Генерация ключей RSA (512 бит)...
Генерация первого простого числа p... готово
Генерация второго простого числа q... готово
Вычисление приватного ключа... готово
Ключи успешно сгенерированы!
```

Рисунок 2 – Генерация ключей для RSA-шифрования



```
=== Сохранение ключей ===
Введите имя файла для публичного ключа (без расширения): test_public_key_1
Введите имя файла для приватного ключа (без расширения): test_private_key_1
Ключи сохранены:
  Публичный: out/test_public_key_1.key
  Приватный: out/test_private_key_1.key
```

Рисунок 3 – Именованение и сохранение ключей

```
=== RSA Шифрование ===
1. Сгенерировать ключи (512 бит)
2. Загрузить ключи из файлов
3. Сохранить ключи в файлы
4. Зашифровать файл
5. Расшифровать файл
6. Показать публичный ключ
0. Выход
> 2

=== Загрузка ключей ===
Введите имя файла публичного ключа (без пути, в директории out/): frst_pub.key
Введите имя файла приватного ключа (без пути, в директории out/): frst_prvt.key
Ключи успешно загружены
```

Рисунок 4 – Загрузка ключей для использования при шифровании

```
=== RSA Шифрование ===
1. Сгенерировать ключи (512 бит)
2. Загрузить ключи из файлов
3. Сохранить ключи в файлы
4. Зашифровать файл
5. Расшифровать файл
6. Показать публичный ключ
0. Выход
> 4

=== Шифрование файла ===

Доступные файлы:
  1. .DS_Store
  2. decrypted_test.txt
  3. test.txt

Выберите файл для шифрования (введите номер от 1 до 3, 0 для отмены): 3

Шифрование: test.txt -> test.txt.enc
Шифрование файла...
Размер файла: 11 байт
Размер блока: 63 байт
Обработано блоков: 1/1
Шифрование завершено успешно!
Зашифрованный файл сохранен: out/test.txt.enc
```

Рисунок 5 – Шифрование файла

```

=== RSA Шифрование ===
1. Сгенерировать ключи (512 бит)
2. Загрузить ключи из файлов
3. Сохранить ключи в файлы
4. Зашифровать файл
5. Расшифровать файл
6. Показать публичный ключ
0. Выход
> 5

=== Расшифровка файла ===

Доступные файлы:
  1. test.txt.enc

Выберите файл для расшифровки (введите номер от 1 до 1, 0 для отмены): 1

Расшифровка: test.txt.enc -> decrypted_test.txt
Расшифровка файла...
Размер зашифрованного блока: 64 байт

Расшифровка завершена успешно!
Расшифрованный файл сохранен: out/decrypted_test.txt

```

Рисунок 6 – Расшифровка файла

Name	Date Modified	Size	Kind
decrypted_test.txt	Today at 22:53	11 bytes	Plain Text
frst_prvt.key	29 Sep 2025 at 10:39	262 bytes	Keynote
frst_pub.key	29 Sep 2025 at 10:39	139 bytes	Keynote
test_private_key_1.key	Today at 22:39	262 bytes	Keynote
test_public_key_1.key	Today at 22:39	139 bytes	Keynote
test.txt	29 Sep 2025 at 10:37	11 bytes	Plain Text
test.txt.enc	Today at 22:52	68 bytes	Document

Рисунок 7 – Состояние директории после выполненных действий



Рисунок 8 – Содержимое изначального и расшифрованного файлов

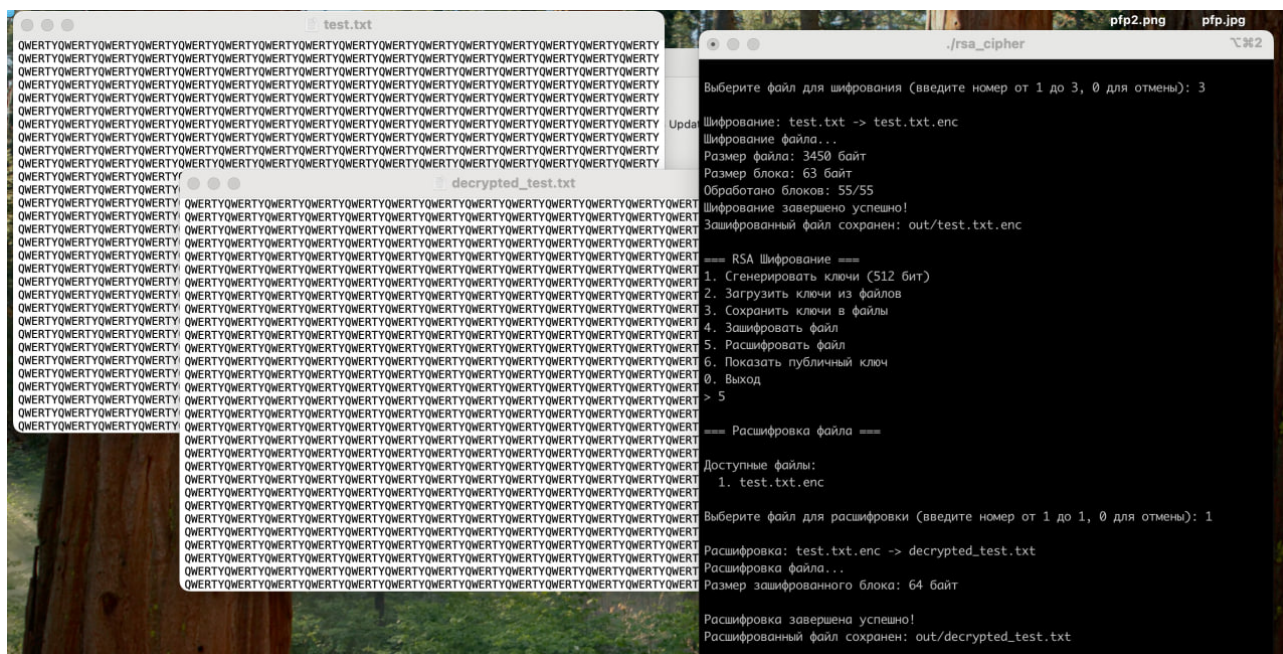


Рисунок 9 – Шифрование большого файла

Также была предусмотрена обработка большинства ошибочных ситуаций без экстренного завершения программы:

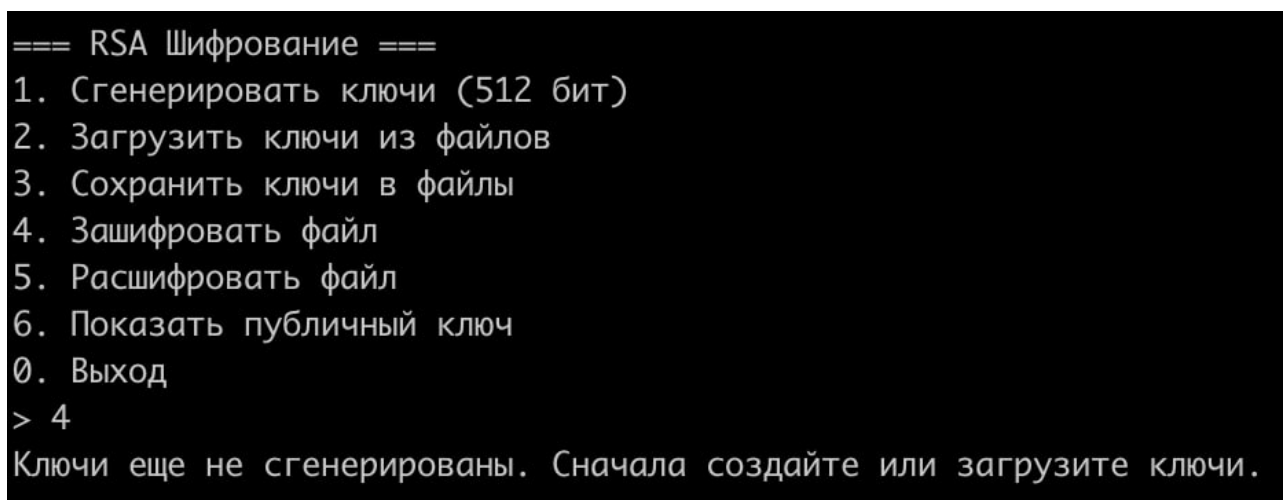


Рисунок 10 – Обработка ошибочных ситуаций